Lab: Create a Sophisticated Webpage

We have covered alot this semester including HTML, basics of visual styling with CSS and using CSS to layout webpages. In this lab you will combine all of these skills to create a webpage for a clothing company called *OurStory*.

Overview

Take a moment to look over the *mockup.pdf* for the webpage. The webpage has a header section that includes the name of company, a navigation bar and then user interface elements like a search feature and account feature on the right.

In the main area there is a large area called a splash area. It has a bright yellow image and text about the company.

Following that are policies about the company including shipping, payment methods and returns.

Finally there are sub-splash areas that are laid out in a two column format. These are meant to intrigue the audience and get them to click on the buttons that take them to future subsequent webpages.

Creating a complex webpage like this is not easy, so we'll do it in steps. We will follow the general steps in the order below:

- HTML Text
- HTML Images
- HTML Layout
- CSS Text
- CSS Backgrounds
- CSS Boxmodel
- CSS Layout

You do not have to follow the method in this lab, this lab is really just discussing layout techniques. How you decide to code your webpage is up to you as long as it follows the layout shown in *mockup.psd*

Note that your HTML / CSS will most likely not match the Mockup.psd 100% and that is okay. It does need to be pixel perfect, but should match the general layout and style of the mockup.

Begin Coding

Start by creating a new HTML document. Add in all the standard HTML basic page tags including: *<!doctype html>*, *<head>*, *<title>*, and *<body>*.

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>OurStory</title>
6  </head>
7  <body>
8
9  </body>
10 </html>
```

© David Hurwich

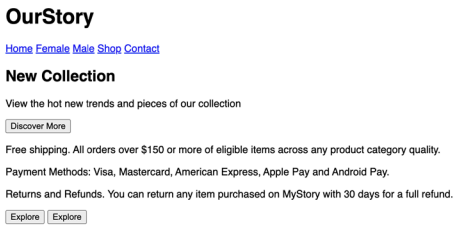
Save it into the same root older that stores all of the photos you'll need for this assignment. Save as *OurStory.html*.

Layout the Text / Hyperlinks

Use website-text.txt to assist you in laying out the text for the webpage.

- The name of the company, *OurStory*, should go into a primary heading.
- Each item in the navigation item should be in an *<a>* tag. You can set *src="#"* to make the links dummy links.
- *"New Collection*" should be an h2.
- Discover More should be *<button>* tag.
- Each policy should be it's own *<p>*

Explore" should be a *<button>*



Save and preview and your webpage similar to the image on the right.

Layout Images / Icons

Now that we've laid out text let's start layout out the images. For this project the images will be applied as background images using CSS or as icons. Let's use Font Awesome to apply these icons.



Open the *code* document into your code editor. It contains the CDN link to Font Awesome. Copy and paste this into the <head> section of the webpage. This should install the Font Awesome component into your project.

Go to Font Awesome's website and find the following icons:

- magnifying-glass
- user
- bag-shopping

Copy the HTML code and paste the three items after the navigational links. Use Preview.pdf as a reference.



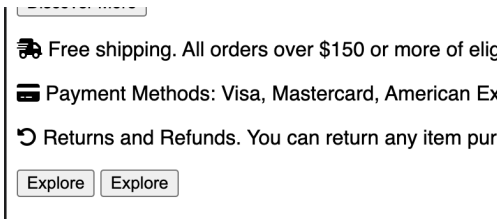
Save and your webpage should look like the image on the right.

Save and preview: Compare your webpage with the image on the right.

© David Hurwich

We will also add icons before each of the policies. Before the text about shipping add the *truck* icon. Before the text about payment use the *credit-card* icon and before the text about the return policy use the *rotate-left* icon.

Save and preview. Your webpage should look roughly like the image on the right.



Create HTML Structure

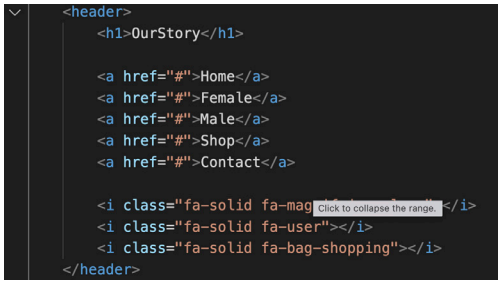
Now that we have text and images laid out, let's begin to organize our page a bit more with HTML structure.

Let's begin by dividing our page into three major categories *<header>*, <main> and <footer>. Remember that the <header> contains all the elements that typically go on the top of every webpage.



For this website the header elements including the:

- company name
- navigation elements
- user interface elements



Next let's lay out the *<main>*
Remember that the *<main>* contains the main part of the webpage. For this design the main will include:

- splash area about the new collection
- policies of the store
- clothes areas for the user to click on

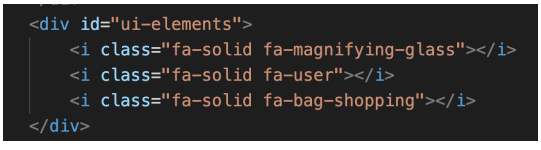
Finally, let's lay out the *<footer>*. Put the copyright and legal information inside of the *<footer>*.

Additonal HTML5 Container Structure

Now that we have a basic structure, let's add additional HTML5 containers to assist the layout.

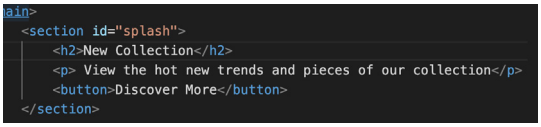
Put all of the navigational links into a *<nav>*

Put the company name, navigation links and user-inferface element each into their own * * tag. Give each an id to make it easier to edit with CSS later. Use the following:



#company-name
#navigation
#ui-elements

Place all of the text related to the splash area inside a *<section>* tag with an id of *#splash*.



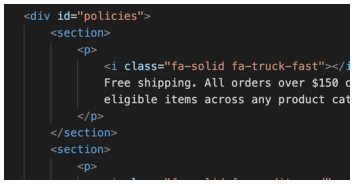
Place each policy into a *<section>* tag. Put all three policy

© David Hurwich

<section> inside another * * tag and call it *#policies*.

Place each *"explore"* button into a *<section>* tag. Each section is an area with clothes the user can click on. The first section should have an id of *#clothes-explore1* and the second will have an id of *#clothes-explore2.* Put the explore sections inside a container with an id of *#clothes.*

Place all of the main HTML containers *<header>*, *<main>* and *<footer>* into a master container * * with an id of *#container.*



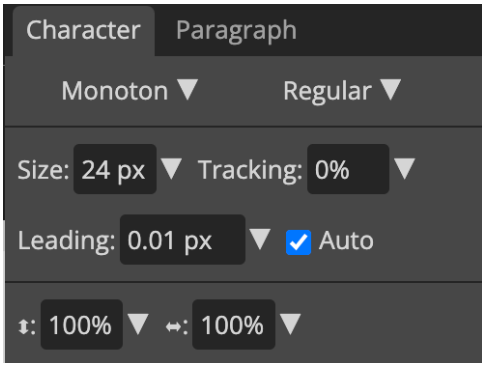


Add Style

Let's begin adding styles to this webpage. In your coding program create a stylesheet and name it *'style.css'*. Make sure to save it to the save folder as images and HTML file.

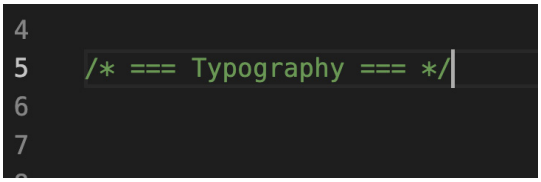
Take a moment to open *Mockup.psd* into PhotoPea or in Adobe Photoshop if you have access to it. Use the tools such as the eye dropper and *Character / Paragraph* panel a better sense of the typography and colors for the project.

You'll need this information later when we add CSS code.



Stylize the Typography

Since we are going to have a lot of CSS for this lab, let's organize our CSS document into sections. It will make it easier to read and edit later on.



Begin by writing the following into your CSS document:

```
/* === Typography === */
```

Let's set up the base text. Remember that with CSS parent tags typically pass CSS properties to their child tags. So when we can use this to our advantage. If we apply a CSS typographical setting to the <body> it will apply to everything inside the <body>, or basically everything in the webpage.

Apply the following:

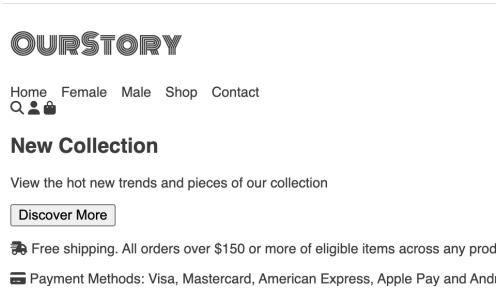



```
body {
    color: #3c3c3c;
    font-size: 10pt;
    line-height: 14pt;
    font-family: "Arial", sans-serif;
}
```

© David Hurwich

Use *Mockup.psd* to assist you with setting the typographical properties for

- Navigation links
- Heading 1
- Heading 2
- Footer
- Buttons



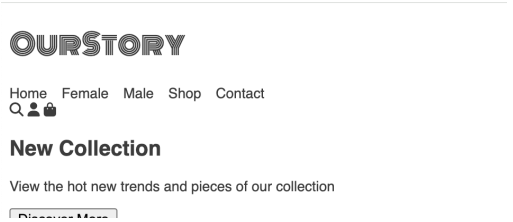
Web Fonts

If you view *mockup.psd* you can see that the title of *OurStory* is a special font called '*Monoton*'. Monoton is a web font that can be found on Google Fonts.

Go to Google Fonts and search for Monoton. Embed the font using the *<link>* and adding it to your HTML document. You can now call the font in your CSS.

Save and preview, your webpage should look roughly like the one on the right.





Stylize the Backgrounds

Add the following code to your CSS document:

```
/* === Backgrounds === */
```

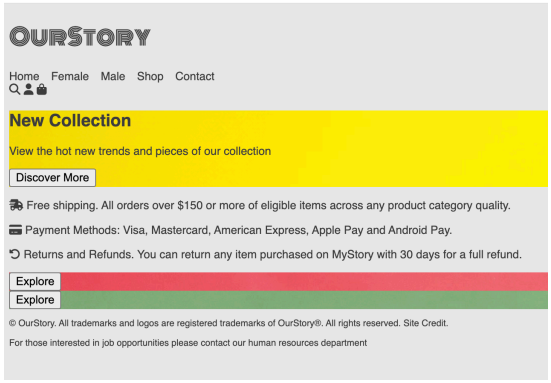
Use *Mockup.psd* to assist you with setting the background colors for the entire webpage or *<body>* and the container or *#container*.

Sample the color using the eyedropper tool and use *background-color* to set the colors in your CSS.



Let's set the background image to the splash area. To do this use *background-image* and *background-size:cover* to apply to the *#splash*. Don't worry that it looks a bit different than the mockup as we'll add dimensions later. Repeat the process for *#clothes-explore1* and *#clothes-explore2*

Save and preview. Your webpage should look roughly like the one on the right.



Setup Box Model Properties

Add the following code to your CSS document:

```
/* === Box Model === */
```

This is where we'll add box model elements such as width,height, padding and margins to our page.

Add a height of 500px to *#splash*

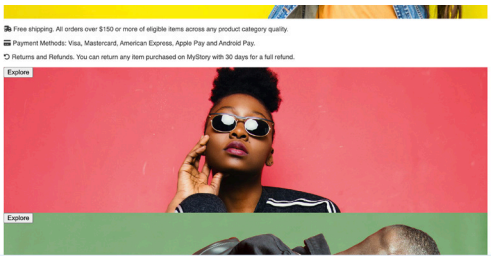
© David Hurwich

Add a height of 300px to *#clothes1* and *#clothes2*

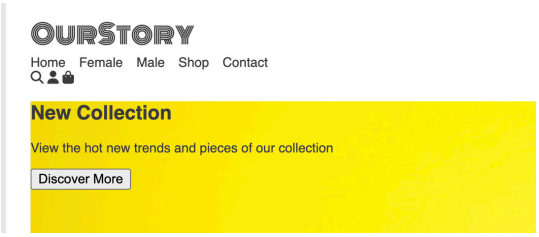
Set a width to the *#container* to *1000px*.

Add padding to the *#container.*

Add margin to the top and bottom of the #policies area



Remove the default padding and margin from the h1 by using the following code:




```
h1 {
   margin: 0px;
   padding: 0px;
}
```

Add Layout to your Webpage

Okay, now that we have a general setup, let's begin to layout items in our page. This means creating columns and generally moving large page blocks arounds.

Add the following code to your CSS document:

```
/* === Layout === */
```

CSS Grids

Let's lay out the columns for the elements in the *<header>*. This will be a three column layout. There are different techniques to achieve this, but we will use CSS grids, as it is one of the easist methods.

As discussed in class examples and previous labs, CSS grids allows you to turn a HTML tag into a grid of rows and columns. Once a tag has been transformed into a grid, it's direct children become the columns.

In this lab we'll begin with the *<header>* tag. If you look at mockup.psd you can see a three column layout.

Begin by setting *<header>* to a CSS grid

```
header {
   display: grid;
}
```

© David Hurwich

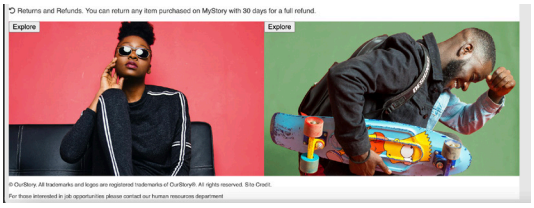
Next use *grid-template-columns* to create columns. Each column should be the same width or in other words each column should be 33%. To set each column equal and allow for padding to be used lets measure columns are fractions or *fr*. Since each column is the same the measurement should be *1fr 1fr 1fr*.

```
header {
   display: grid;
   grid-template-columns: 1fr 1fr 1fr;
}
```



Repeat the process for the *#clothes* ares as well. Create that as a 50/50 two column split using fractions.

Your layout should look like the image on the right.



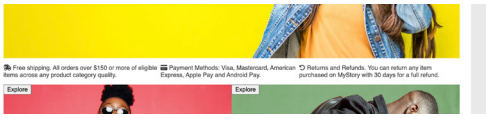
Flexbox

Flexbox is another way to layout a sophisticated webpage. We can use flexbox to create columns, but also use it to place items horizontally and vertically as well.

Like with CSS grids you can turn a container HTML to display as a flexbox. When you do this all of the direct children of the flexbox become ***flex items***. As flex items they can be positioned horizontally and vertically.

Let's use flexbox to turn the *#policies* area into three columns.

Select *#policies* and turn it into a flexbox



```
#policies {
   display: flex;
}
```

Save and you will see that each policy becomes a column. Flexbox lays out each <section> horizontally to will fill the horizontal space. These columns are rather tight as there is quite a bit of text. You can put space between by using the CSS property gap.



```
#policies {
   display: flex;
   gap: 20px;
}
```

Flexbox Horizontal / Vertical Position

Flexbox can also be used to position flex items horizontally and vertically in relation to it's parent. If you look at *mockup.psd* you can see the explore button above the woman in track suit should be in the vertical and horizontal center, this can easily be done

© David Hurwich

using flexbox.

Set *#clothes-explore1* to being a flexbox.

```
#clothes-explore1 {
    display: flex;
}
```

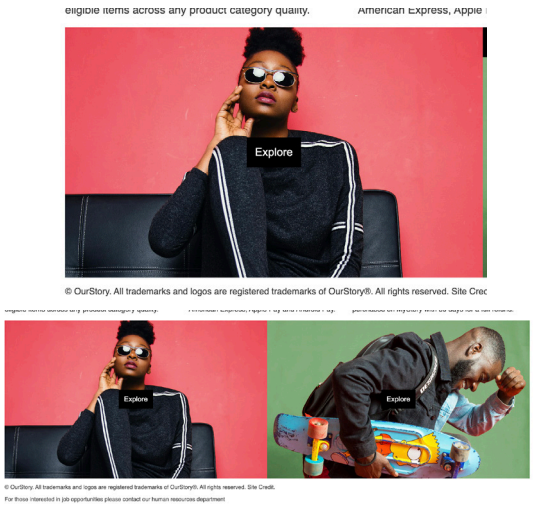
This will make the *<section>* the flexbox and the button the flex item. Now we can use *align-items* to align the button to the vertical center. align-items displays items perpentular to the direction of the flexbox. By default flexboxes are horizontal or rows, that means align-items will affect the vertical value of the flex item.

```
#clothes-explore1 {
    display: flex;
    align-items: center;
}
```

align-items only affects the vertical, not horizontal placement. We can use *justify-content: center* to place the flex item in the horizontal center. justify-content displays items with the direction of the flexbox, since this is row, it will present them horizontally.

```
#clothes-explore1 {
    display: flex;
    align-items: center;
    justify-content: center;
}
```

Save and preview, the button should appear like the image on the right. Repeat the process for button within *#clothes-explore2*.



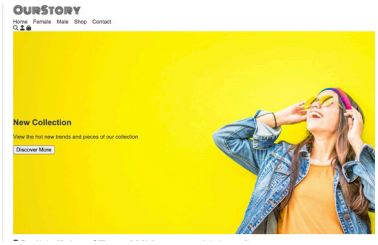
Alternative Vertical Alignment Method

Sometimes we do not necessarily want to use flexbox to vertically align something. For instance, if you were to use display:flex and align-items: center on the *#splash* area you would get something like the following on the right. This is because there are multiple items such as the *<h2>*, *<p>* and *<button>*.

You could solve the issue by changing the flexbox from the default row to a column, but it still requires extra work. An easier method is a newer CSS property called *align-content*. align-content works on any HTML container and allows you to place text vertically.

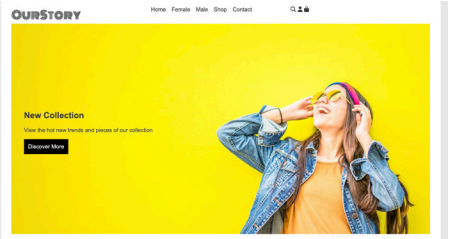
Apply the following to the #splash

```
#splash {
  height: 500px;
  align-content: center;
}
```



The result is the splash text will appear in the vertical center. What is nice about this technique is that it does not require turning the container tag into a grid or flexbox.

To get the text to not be against the wall of the splash area add padding to the left side of #splash. Your document should look roughly like the image on the right when you are done.



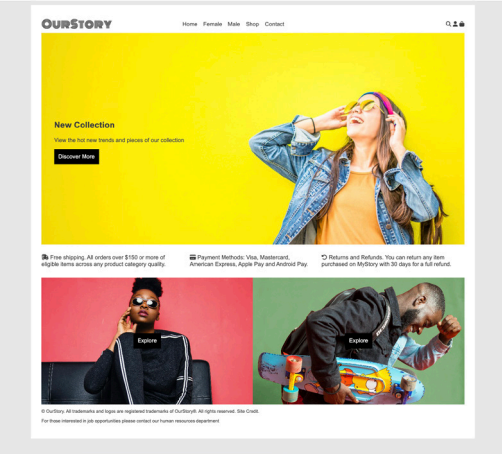
Finishing Touches

We are almost there, but let's add a few more touches to get everything where we want it.

Center the *#container* by using *margin: auto*

Use *text-align* to move the UI elements to the right side of the webpage

Use either flexbox or *align-content* to vertically center the navigation bar and UI elements in the *<header>*



Save and preview and your final webpage should look roughly like the image on the right.

Submission

Whew! That was alot, but hopefully you got the sense of how to put together a sophisticated webpage using HTML and CSS.

Please turn all the related lab files to Canvas as a zipped folder for grading.

© David Hurwich